

Manual MLOps Challenge: Predictive Maintenance System

Name: Nitesh Kumar Shah

ROLL: ID25M806

Course: DA5402 – MLOps

Assignment: A1 – Manual MLOps Challenge

Date: February 2026

15 februari 2026

1. Primary Challenges in Manual Data Versioning

The most demanding part of this assignment was maintaining discipline in handling dataset versions manually. Without automated tools such as DVC, all dataset management responsibilities had to be enforced through careful logging and structured file handling.

1. **Consistent Version Naming:** Establishing and strictly following a clear naming convention (e.g., v1.csv, day2.csv) was essential. Even a minor inconsistency or accidental overwrite could compromise the reproducibility of the entire pipeline.
2. **Manual Lineage Documentation:** Each preprocessing step had to be recorded explicitly in manifest.txt, including the origin of the data, transformation details, script references, and dataset dimensions. Missing a single entry would weaken traceability.
3. **Lack of Automated Rollback:** In contrast to automated version control systems, reverting to an earlier dataset required manual intervention. This exposed how vulnerable manual systems are to human oversight and operational mistakes.

2. Guaranteeing Consistency Between Training and Production Models

Ensuring that the model deployed through the API was exactly the same as the one evaluated during training required deliberate safeguards.

1. **Explicit Metadata Tracking:** After each training run, the model artifact (model_v1.pkl) was accompanied by a structured metadata log recording the dataset used, timestamp, and validation performance. This created a traceable mapping between model and data.
2. **Centralized Configuration Control:** All file paths, parameters, and thresholds were defined in config.yaml. By eliminating hardcoded values, the risk of silent configuration mismatches was reduced.

3. **Deployment State Logging:** Each API launch appended an entry to `deployment_log.csv`, recording which model version was active. This enabled post-deployment verification of the serving model.

3. Verifying Correct Dataset Usage During Training

The configuration file functioned as the authoritative source for selecting dataset versions. The training script dynamically retrieved dataset paths from `config.yaml`, preventing ambiguity.

Additional validation measures included:

1. **Immediate Failure on Missing Files:** If a referenced dataset did not exist, the training process halted with a clear error message rather than proceeding silently.
2. **Cross-Checking Logs:** After training, the dataset recorded in `model_metadata.log` was compared with entries in `manifest.txt` to confirm that the intended preprocessing pipeline had been applied.
3. **Time-Aware Splitting Strategy:** Because the dataset is chronological, the first 7000 records were used for training to reflect realistic deployment conditions.

4. Most Valuable Features of an Automated MLOps Platform

This manual implementation highlighted the importance of automation in production systems. The three most impactful automated capabilities would be:

1. **Integrated Data Version Tracking:** A system similar to DVC that automatically links datasets with corresponding models and maintains lineage without manual file bookkeeping.
2. **Centralized Model Registry:** A registry that stores model artifacts, hyperparameters, and performance metrics in a structured environment, enabling comparison and controlled promotion to production.
3. **Continuous Monitoring and Alerting:** An automated mechanism that evaluates production performance in real time and signals when retraining is required.

5. Incident in Tracking the Active Serving Model

During retraining cycles, a discrepancy emerged between the configured model and the model actively serving predictions.

1. **Issue Encountered:** The configuration file was updated to reference a new model, but the API process was not restarted. As a result, the previous model continued to generate predictions.
2. **Detection:** Performance monitoring on simulated production data revealed unexpected degradation, prompting investigation.
3. **Resolution:** Deployment logging was strengthened and runtime checks were introduced to verify that the loaded model matched the configured model version.

6. Simulating Drift and Implementing Monitoring

To approximate real-world operational conditions, a secondary dataset (day2.csv) was generated from later time segments of the original data. Minor temperature adjustments and Gaussian noise were introduced to represent gradual sensor variation.

A monitoring script evaluated model predictions on this modified dataset and compared error metrics with baseline validation results. If the observed deviation exceeded a predefined threshold from config.yaml, a retraining notification was issued.

This process completed the operational loop:

Train → Deploy → Evaluate → Detect Deviation → Retrain.

Conclusion

This assignment demonstrated that building a machine learning model is only a fraction of the overall challenge. Managing data versions, maintaining deployment integrity, and detecting performance drift require structured operational practices.

Manual implementation revealed the fragility and cognitive burden associated with non-automated workflows. Tools such as DVC, MLflow, and orchestration frameworks exist precisely to mitigate these risks. By recreating these mechanisms manually, the necessity of automation in scalable ML systems becomes evident.

AI Disclosure Appendix

In accordance with the Code of Conduct for Fair Use of AI, the following AI assistance was used in this assignment:

Activity	AI Usage
Boilerplate Code	Used AI assistance for structuring FastAPI endpoints (including the <code>/predict</code> endpoint) and for minor debugging support.
Debugging	Used AI to clarify scikit-learn API usage and resolve pickle serialization issues during model saving and loading.
Concept Explanation	Used AI to better understand theoretical concepts related to distribution shift and drift detection.
Documentation	Used AI to refine language, improve clarity, and structure this written report.
Core Logic	NOT USED – All manual data versioning strategies (including <code>manifest.txt</code> creation and logging logic), model registry design, deployment tracking, monitoring scripts, drift simulation logic, and retraining triggers were independently implemented.

All architectural decisions, manual MLOps strategies, monitoring mechanisms, dataset version tracking, and problem-solving approaches were developed independently. AI tools were used strictly as a productivity aid for scaffolding and language refinement, not for core system design.